

Template Method Design Pattern

The **Template Method Pattern** is a **behavioral design pattern** where you define the **overall algorithm/steps in a parent class**, but allow subclasses to provide their own implementation for certain steps.

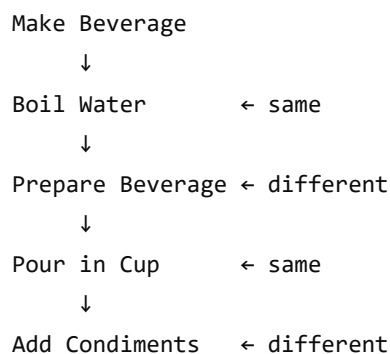
Simple idea

Think of making tea and coffee:

Both follow the same overall process:

1. Boil water
2. Prepare the drink
3. Pour into cup
4. Add something extra

The **overall process is fixed**, but some steps differ.



```
graph TD; A[Make Beverage] --> B[Boil Water]; B --> C[Prepare Beverage]; C --> D[Pour in Cup]; D --> E[Add Condiments]; B -.-> B1[← same]; C -.-> C1[← different]; D -.-> D1[← same]; E -.-> E1[← different];
```

That's exactly what Template Method does.

Java Example

```
abstract class Beverage {  
  
    // Template method  
    public final void prepare() {  
        boilWater();  
    }  
}
```

```
        prepareBeverage();
        pourInCup();
        addCondiments();
    }

    private void boilWater() {
        System.out.println("Boiling water");
    }

    protected abstract void prepareBeverage();

    private void pourInCup() {
        System.out.println("Pouring into cup");
    }

    protected abstract void addCondiments();
}
```

Now create Tea:

```
class Tea extends Beverage {

    @Override
    protected void prepareBeverage() {
        System.out.println("Steeping tea");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Adding sugar and lemon");
    }
}
```

Coffee:

```
class Coffee extends Beverage {

    @Override
    protected void prepareBeverage() {
        System.out.println("Brewing coffee");
    }

    @Override
    protected void addCondiments() {
        System.out.println("Adding milk and sugar");
    }
}
```

}

</> Java



Usage:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Beverage tea = new Tea();  
        tea.prepare();  
  
        System.out.println();  
  
        Beverage coffee = new Coffee();  
        coffee.prepare();  
  
    }  
}
```

Output:

```
Boiling water  
Steeping tea  
Pouring into cup  
Adding sugar and lemon  
  
Boiling water  
Brewing coffee  
Pouring into cup  
Adding milk and sugar
```



What is important here?

The parent class controls the **sequence**:

```
public final void prepare() {  
    boilWater();  
    prepareBeverage();  
    pourInCup();  
    addCondiments();  
}
```

The subclasses control only the **variable steps**:

```
prepareBeverage()
```

```
</> Java
```



So:

Template Method = fixed algorithm structure + customizable individual steps

Why `final` is commonly used?

Notice:

```
</> Java
```



We don't want a subclass to change the algorithm:

```
Boil water
  ↓
Prepare
  ↓
Pour
  ↓
Condiments
```



For example, we don't want:

```
Prepare
  ↓
Boil
  ↓
Condiments
  ↓
Pour
```



The parent class owns the **workflow**.

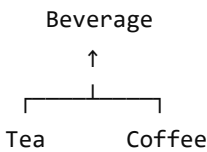
The child class owns the **specific implementation**.

Template Method vs Strategy

This is an important interview distinction.

Template Method

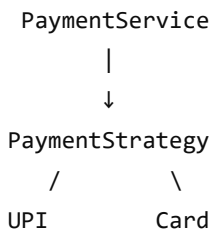
Uses **inheritance**:



Parent defines the algorithm, children customize steps.

Strategy

Uses **composition**:



You can change the strategy at runtime.

Easy way to remember

Template Method	Strategy
Inheritance	Composition
Parent controls algorithm	Context delegates algorithm
Some steps overridden	Entire behavior can be replaced
Usually fixed workflow	More runtime flexibility

Real-world Java/Spring examples

You will encounter this concept in frameworks frequently.

For example, Spring's template-style classes:

```
JdbcTemplate
RestTemplate
```

</> Java



The general idea is:

Framework controls common workflow



You provide the variable part



For example, conceptually:

```
jdbcTemplate.query(..., resultSet -> {  
    // Your custom processing
```

</> Java



The framework handles common JDBC work such as connection/resource management, while you provide the application-specific part.

Interview definition

If an interviewer asks "What is Template Method Design Pattern?", you can say:

Template Method is a behavioral design pattern where a superclass defines the skeleton of an algorithm and lets subclasses customize specific steps without changing the overall algorithm structure.

The 3 things you should remember are:

1. Superclass defines the algorithm
2. Some steps are customizable by subclasses
3. Overall sequence remains controlled by the superclass



anything more 7-8 year experienced developer need to learn?

Yes. For a **7–8 year experienced Java developer**, knowing only the basic Template Method definition is not enough. You should understand these areas, but you don't need to go extremely deep into obscure theory.

Template Method — interview-focused scope

1. Core concept — Must know

- Template Method definition
- Why it is a behavioral pattern
- Algorithm skeleton
- Fixed steps vs variable steps
- Abstract methods / concrete methods
- Why the template method is often `final`

2. Hook methods — Must know

A **hook** is an optional method that subclasses can override.

</> Java



```
abstract class DataProcessor {

    public final void process() {
        readData();
        processData();

        if (shouldValidate()) {
            validate();
        }

        saveData();
    }

    abstract void readData();

    abstract void processData();

    void validate() {
        System.out.println("Default validation");
    }

    boolean shouldValidate() {
        return true;
    }

    void saveData() {
        System.out.println("Saving data");
    }
}
```

Here:

</> Java

```
boolean shouldValidate()
```



✦ Upgrade



is a **hook**.

A subclass can change it:

</> Java

```
@Override
boolean shouldValidate() {
    return false;
}
```



So remember:

Abstract method = subclass must implement.

Hook method = subclass may override.

3. Hollywood Principle — Good to know

Template Method follows the idea:

"Don't call us, we'll call you."

The superclass controls the flow and calls subclass methods when required.

Client



templateMethod()



Superclass controls workflow



Subclass-specific methods



You don't normally call the individual steps yourself.

4. Template Method vs overriding the whole method — Must know

Bad design:

</> Java

```
class Tea extends Beverage {
```



```
@Override
public void prepare() {
    // completely different algorithm
}
}
```

This allows the subclass to change the entire workflow.

Template Method prevents this:

```
</> Java

public final void prepare() {
    boilWater();
    prepareBeverage();
    pourInCup();
}
```

Only the intended extension points can change.

5. Template Method vs Strategy — Very important

You should be able to explain this confidently.

Template Method:

```
Inheritance
  ↓
Parent defines algorithm
  ↓
Child customizes steps
```

Strategy:

```
Composition
  ↓
Context delegates behavior
  ↓
Strategy can be replaced
```

A common interview question is:

"When would you choose Template Method over Strategy?"

Answer:

Use Template Method when the **overall algorithm should remain fixed** and only certain steps vary.

Use Strategy when you want to **replace the behavior/algorithm itself**, especially at runtime.

6. Template Method vs Factory Method — Good to know

They are different patterns, although they are often used together.

Template Method:

Controls algorithm



Factory Method:

Controls object creation



Example:

`</> Java`



```
public final void process() {  
    Repository repository = createRepository();  
    repository.save();  
}
```

Here `process()` could be the Template Method, while `createRepository()` could be a Factory Method.

7. Real Spring/framework examples — Must know conceptually

You should recognize template-style designs in frameworks such as:

- JdbcTemplate
- RestTemplate
- TransactionTemplate

But don't waste time memorizing their internal source code.

Understand the **problem they're solving**:

Common boilerplate



Framework/template handles it





Developer provides custom operation

8. Advantages and disadvantages — Must know

Advantages

- Eliminates duplicated algorithm structure
- Enforces consistent workflow
- Allows controlled customization
- Promotes code reuse

Disadvantages

- Relies on inheritance
- Can create tight coupling between parent and subclasses
- Too many hooks can make the design complicated
- Changing the parent algorithm can affect many subclasses

What I would consider sufficient for 7–8 years

Topic	Priority
Template Method concept	● Must know
Algorithm skeleton	● Must know
Abstract vs concrete steps	● Must know
Hook methods	● Must know
<code>final</code> template method	● Must know
Hollywood Principle	● Good
Template vs Strategy	● Must know
Template vs Factory Method	● Good
Spring/framework examples	● Must know conceptually
Advantages/disadvantages	● Must know
Source-code implementation of frameworks	● Not necessary

You don't need to go deeper than this for interviews. The most important part for your experience level is being able to look at a real codebase and recognize: **"The workflow is fixed, these steps vary → Template Method is being used."**



is this sufficient?

Yes. For a 7–8 year experienced Java/Spring developer, this is sufficient for Template Method Design Pattern.

You should be able to:

- Explain the pattern clearly
- Implement it in Java
- Explain **template method vs hook method**
- Explain why the template method is often `final`
- Explain **Hollywood Principle**
- Compare **Template Method vs Strategy**
- Compare **Template Method vs Factory Method**
- Identify template-style designs in Spring/frameworks
- Explain advantages and disadvantages
- Give a practical use case

One final thing to practice

Be ready for a scenario question:

"You have 5 different file processors. All follow the same sequence — validate → read → transform → save — but the implementation of each step differs. How would you design it?"

If you can design that using Template Method and explain **why**, you're at the right level.

No need to spend more time on Template Method. Move to the next design pattern.

